

Rapport TP3

LOG8235 - Agents intelligents pour jeux vidéo

Equipe

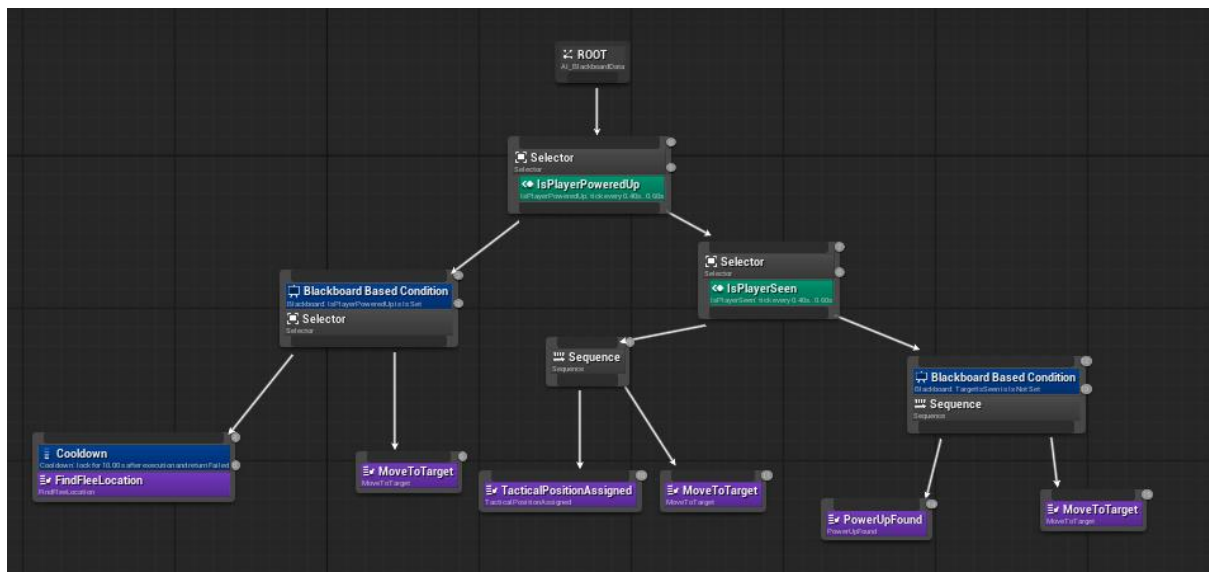
Baptiste Leducq- 2486057

Guillaume Lesourd - 2488094

Mauricio Medina - 1537659

Noah Mesple - 2487453

Utilisation d'un behavior tree



Le Behavior Tree utilise deux séquences pour déterminer le comportement à adopter. Le service `IsPlayerSeen` permet de vérifier si le joueur est dans le champ de vision de l'agent. Si le joueur est vu, l'agent est assigné à un groupe de poursuite qui tente d'encercler le joueur. L'information de vision du joueur est partagé à tous les agents à l'aide d'un broadcast. Chaque agent obtient une position tactique pour encercler le joueur. Si le joueur n'est pas vu, l'agent va chercher un collectible à récupérer grâce à la Task `PowerUpFound`. La Task `MoveToTarget` permet à l'agent de se déplacer vers la destination qu'il a obtenue en fonction de son comportement.

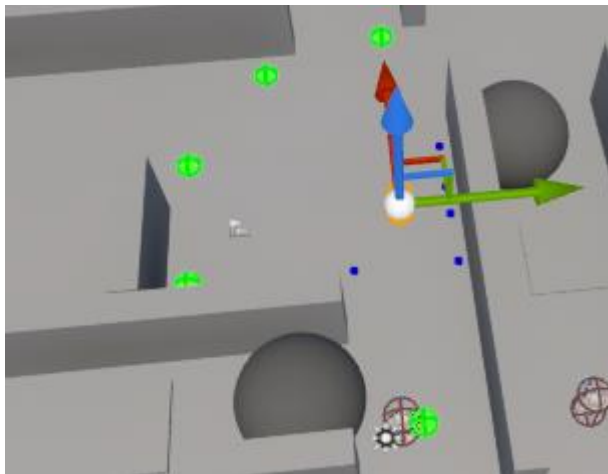
2. Création de groupe de poursuite

Notre AI master est dans le gamemode, accesible par tout:

```
#include "SoftDesignTrainingGameMode.h"
```

```
SDTAIController.cpp  SoftDesignTr...GameMode.cpp
SoftDesignTraining  (Global Scope)
175  if (losHit.GetComponent()->GetCollisionObjectType() == COLLISION_PLAYER)
176  {
177      hasLosOnPlayer = true;
178  }
179  }
180
181  if (hasLosOnPlayer)
182  {
183      if (GetWorld()->GetTimerManager().IsTimerActive(m_PlayerInteractionNoLosTimer))
184      {
185          GetWorld()->GetTimerManager().ClearTimer(m_PlayerInteractionNoLosTimer);
186          m_PlayerInteractionNoLosTimer.Invalidate();
187          //DrawDebugString(GetWorld(), FVector(0.f, 0.f, 10.f), "Got LoS", GetPawn(), FColor::Red, 5.f, false);
188      }
189
190      ASoftDesignTrainingGameMode* GM = GetWorld()->GetAuthGameMode<ASoftDesignTrainingGameMode>();
191      if (GM)
192      {
193          GM->PlayerSeenByAI(this);
194      }
195
196      return true;
197  }
198  else
199  {
200      if (!GetWorld()->GetTimerManager().IsTimerActive(m_PlayerInteractionNoLosTimer))
```

Quand un Bot voi le joueur , il informe le gamemode line 193. Dans le Gamemode on vérifie si la liste du group et vide pour faire un seul requête de EQS autour du joueur.



```

193 void ASoftDesignTrainingGameMode::PlayerSeenByAI(ASDTAIController* InstigatorAIController)
194 {
195     // Prevent creation when the Bots are in flee mode. The raycast can still detect the player
196     // when moving to flee position
197
198     if (SoftPlayerController)
199     {
200         if (ASoftDesignTrainingMainCharacter* Player = Cast<ASoftDesignTrainingMainCharacter>(SoftPlayerController->GetCharacter())
201         {
202             if (Player->IsPoweredUp())
203             {
204                 //UE_LOG(LogTemp, Warning, TEXT("=== AIControllers ==="));
205                 return;
206             }
207         }
208     }
209
210     if (AIControllersTacticalGroup.IsEmpty())
211     {
212         RunEQSForTacticalGroupCreation();
213     }
214     else
215     {
216         //UE_LOG(LogTemp, Warning, TEXT("AIControllersTacticalGroup.Contains(InstigatorAIController)"));

```

A la reponse:

void

ASoftDesignTrainingGameMode::OnQueryTacticalPositionsAttack(UEnvQueryInstance
BlueprintWrapper* QueryInstance, EEnvQueryStatus::Type QueryStatus)

On attribue un point pour chaque Bot le plus proche du jouer. Et on garde aussi le
PlayerLKP.

3. Comportement de poursuite en groupe

Dans le tick du game mode on met a' jours les points avec un nouveau EQS si jamais le
joueur arrive a' s'en aller. On accept aussi de nouveau Bots a se rejoindre au group.

```

void ASoftDesignTrainingGameMode::Tick(float DeltaSeconds)
{
    Super::Tick(DeltaSeconds);

    if (AIControllersTacticalGroup.IsEmpty())
        return;

    if (!SoftPlayerController)
        return;

    //Update Tactical group
    if (FVector::DistSquared(PlayerLKP, SoftPlayerController->GetPawn()->GetActorLocation()) > 250000.0)
    {
        RunEQSForTacticalGroupCreation();

        PlayerLKP = SoftPlayerController->GetPawn()->GetActorLocation();
    }

    for (ASDTAIController* AIController : AIControllersTacticalGroup)
    {
        FVector npcPosition = AIController->GetPawn()->GetActorLocation();
        FVector npcHead = npcPosition + FVector::UpVector * 200.0f;
        UWorld* npcWorld = GetWorld();

        DrawDebugSphere(npcWorld, npcHead, 20.0f, 32, FColor::Magenta);
    }

    // Your logic here
}

```

Quand le joueur prend le power up, le group se efface.

```
void ASoftDesignTrainingGameMode::PlayerPickUpPowerUp()
{
    for (ASDTAIController* AIController : AIControllersTacticalGroup)
    {
        AIController->SetIsTargetPlayerSeen(false);
    }

    AIControllersTacticalGroup.Empty();

    //OnPlayerPowerUp.Broadcast(true);
}
```

```
void ASDTCollectible::Collect(ASoftDesignTrainingCharacter* InstigatorCharacter)
{
    if (ASoftDesignTrainingMainCharacter* Player = Cast<ASoftDesignTrainingMainCharacter>(InstigatorCharacter))
    {
        ASoftDesignTrainingGameMode* GM = GetWorld()->GetAuthGameMode<ASoftDesignTrainingGameMode>();
        if (GM)
        {
            GM->PlayerPickUpPowerUp();
        }
    }

    GetWorld()->GetTimerManager().SetTimer(m_CollectCooldownTimer, this, &ASDTCollectible::OnCooldownDone, m_CollectCooldownDuration, false);

    GetStaticMeshComponent()->SetVisibility(false);
}
```

Dans le behavior tree la branch la plus a' gache represente le Flee

